PYTHON DATA PROCESSING AND API INTEGRATION TOOLKIT

Technology: Python 3.12

Testing Framework: Pytest

Development Tool: Visual Studio Code

Project Date: 23 October 2026

Table of Contents

1. Introduction
2. Project Objectives
3. Technologies Used
4. Project Structure
5. System Architecture
6. CSV Data Processing
7. API Integration
8. Error Handling and Logging
9. Automated Testing Using Pytest
10. Installation and Execution
11. Generated Outputs
12. Recovery and Troubleshooting
13. Security and Reliability
14. Limitations and Future Enhancements
15. Learning Outcomes
16. Conclusion

1. Introduction

The Python Data Processing and API Integration Toolkit is a command-line application developed using Python. It processes CSV files, validates records, identifies invalid data, communicates with an external API, and generates reports.

The application follows a modular programming approach. Each module performs a specific task, making the project easier to understand, test, maintain, and extend. The project uses CSV and JSON file handling, API integration, exception handling, logging, and automated testing.

2.Project Objectives

The main objective is to develop a reliable Python application for data processing and API integration.

- Read and process CSV files.
- Validate input data and identify invalid records.
- Detect duplicate IDs and negative amounts.
- Handle missing values and invalid input.
- Integrate an external API using Python.
- Handle API failures and timeouts.
- Generate JSON reports and application logs.
- Test functionality using pytest.
- Run automated tests without depending on a live external API.

3.Technologies Used

Technology	Purpose
Python 3.12	Application development
CSV	Input data processing
JSON	Report and API response storage
Requests	API communication
Pytest	Automated testing
Logging	Recording application events
Visual studio code	Code development and execution
Git and GitHub	Version control and project hosting

4.Project Structure

Final_Mini_Project/

├── src/

│ ├── init.py

│ ├── processor.py

processor.py: Reads CSV files, validates records, detects invalid data, and processes valid records.

api_client.py: Sends API requests and handles connection failures, timeouts, and response errors.

main.py: Coordinates the workflow, processes command-line arguments, configures logging, and generates reports.

test_processor.py: Tests CSV processing, data validation, missing files, and missing columns.

test_api_client.py: Tests successful API responses, timeouts, and connection failures using mocks.

test_integration.py: Tests the complete application workflow.

requirements.txt: Lists the external Python dependencies.

README.md: Provides project information, installation, execution, and testing instructions.

Excludes virtual environments, Python cache files, and generated reports from version control.

5.System Architecture

The application follows a modular architecture:

Input CSV File → Main Application → CSV Processor and API Client → Reports and Logs

- The application reads records from the input CSV file.
- The main application coordinates the processing workflow.
- The CSV processor validates input records.
- Valid records are processed; invalid records are recorded with rejection reasons.
- The API client retrieves data from an external API.
- The application generates JSON reports and records events through logging.
  This architecture separates responsibilities and makes individual modules easier to test and maintain.

6.CSV Data Processing

CSV stands for Comma-Separated Values. It is a file format used to store structured data in rows and columns. The input file contains three required fields: id, name, and amount.

```
id,name,amount
101,Divya,1500
102,Ravi,2500
103,Sita,3000
102,Kiran,1800
104,Arun,-500
105,Meena,1200
106,,900
```

Data Validation Rules

• Required columns must be present.
• Required field values must not be empty.
• Duplicate IDs are rejected.
• Negative amounts are rejected.
• Valid records are processed.
• Invalid records are recorded with their rejection reasons.

Processing Results

For the sample input file, the application produced the following recorded results:
• Total records processed: 7
• Valid records: 4
• Rejected records: 3
The rejected records include a duplicate ID, a negative amount, and an empty required value.

7.API Integration

An API (Application Programming Interface) allows different software applications to communicate. The project uses the Python Requests library to send HTTP requests to an external API.

- Sends requests to the configured API endpoint.
- Uses a timeout to prevent requests from waiting indefinitely.
- Handles connection failures and timeout errors.
- Processes valid API responses.
- Saves API response data separately as JSON.

The configured request timeout is five seconds. During automated testing, mock objects simulate API responses and failures so tests do not depend on an external service being available.

8.Error Handling and Logging

Error Handling
Error handling allows the application to respond appropriately when problems occur.
• Missing input files or required CSV columns.
• Empty required values, duplicate IDs, and negative amounts.
• API connection failures and timeouts.
• Invalid response data.

Logging

Logging records important events during execution, including application execution, processing completion, successful operations, and API failures. Logs support troubleshooting and review of application behavior.

9.Automated Testing Using Pytest

Pytest is a Python testing framework used to verify whether an application behaves as expected.

test_processor.py: Tests CSV processing, valid and invalid records, missing files, and missing columns.
test_api_client.py: Tests successful API responses, timeout errors, and connection failures using mocks.
test_integration.py: Tests the complete workflow, including CSV processing, duplicate detection, API handling, and report generation.

Test Execution Command

python -m pytest tests -v

Test Results

- Total tests: 7
- Passed: 7
- Failed: 0

The project test suite reported seven passing tests. The same suite was also run in a separate virtual environment after installing dependencies from requirements.txt, and all seven tests passed again. Screenshot evidence: Insert docs/test_results.png in this section when preparing the final report.

10.Installation and Execution

Prerequisites

- Python 3.12
- Visual Studio Code or another Python-compatible editor
- Dependencies listed in requirements.txt

Step 1: Install Dependencies python -m pip install -r requirements.txt
Step 2: Run the Application python .\src\main.py
Step 3: Run Automated Tests python -m pytest tests –v

The application processes the input CSV file, validates records, handles API operations, and generates reports. Check the test output to confirm that all seven tests pass. Screenshot evidence: Insert docs/application_output.png in this section.

11.Generated Outputs

summary.json: Contains processing statistics and information about rejected records.
api_response.json: Contains API response data when a successful response is received.

The output files help users review processing results and API responses separately. The generated reports directory is excluded from version control through .gitignore.

12.Recovery and Troubleshooting

Problem	Recommonded solution
Input CSV file is missing	Check the file path and restore the file.
Required columns are missing	Correct the CSV headers.
Records are rejected	Review rejection reason and correct the input data.
API request times out	Check network connectivity and API availability.
Required packages are missing	Install dependencies from requirements.txt.

After correcting a problem, rerun the application or the relevant tests. The current versio P n supports manual recovery and rerunning the application. Automatic retries and automatic restart recovery are not implemented.

13.Security and Reliability

- Input records are validated before processing.
- API requests use a timeout.
- API communication errors are handled.
- Automated tests use mocked API responses.
- Generated reports and virtual environments are excluded from version control.
- Application logs support troubleshooting.
- Processing summaries are stored separately from API responses.

For production use, further improvements could include secure secret management, additional input validation, automatic retries, and more extensive monitoring.

14.Limitations and Future Enhancements

Current Limitations

- The application is a command-line tool without a graphical user interface.
- API response data is stored separately from processed CSV data.
- Automatic retries are not implemented.
- Automatic restart recovery is not implemented.
- The current automated test suite contains seven tests.
- The application does not use a database for persistent record management.

Future Enhancements

- Add automatic retries for temporary API failures.
- Add more test cases for edge conditions.
- Integrate a database.
- Develop a web-based user interface.
- Add continuous integration checks using GitHub Actions.
- Generate more detailed processing reports. • Improve monitoring and structured logging.

15.Learning Outcomes

Through this project, the following technical skills were practised:

- Python modular programming.
- CSV and JSON file handling.
- Data validation and exception handling.
- API integration using Requests.
- Application logging.
- Unit testing and integration testing using pytest.
- Mocking external API dependencies.
- Virtual environment setup and dependency management.
- Technical documentation and version control. This project provides practical experience in developing and testing a modular Python application.

16.Conclusion The Python Data Processing and API Integration Toolkit demonstrates the use of Python for data processing, API communication, input validation, error handling, logging, and automated testing. The modular architecture makes the application easier to understand, maintain, and extend. The automated test suite completed successfully with seven passing tests, including tests for data validation, API failures, and the complete workflow. This project provides practical experience in building a small, testable Python application using common software development practices.

Overall status : Completed successfully